

1. 자연어 처리(NLP)란?

컴퓨터는 텍스트를 직접 이해하는 것이 불가능하다. 따라서 이를 숫자 벡터로 변환하는 과정이 필수적이다. 이 변환을 위한 첫 단계가 토큰화이다.

자연어 처리 과정: 텍스트 입력 -> 토큰화 -> 수치화(벡터) -> 모델 학습

인간 언어는 모호성, 가변성, 구조성을 띤다.

2. 토큰화 핵심 개념

토큰화: 텍스트를 분석 가능한 최소 단위인 토큰으로 분리하는 과정

말뭉치: 분석 대상이 되는 텍스트 데이터셋

토큰: 토큰화된 결과물

토큰나이저: 토큰화를 수행하는 규칙 혹은 모델

3. 토큰나이저 구축 방법

1. 공백 분할: 띄어쓰기 기준
2. 정규 표현식: 패턴 규칙 사용
3. 어휘 사전 기반: 사전에 등록된 단어만 인식 (OOV 문제 발생 가능)
4. 머신러닝 기반: 데이터로부터 규칙을 자동 학습

4. 토큰화 유형별 특징

1. 단어 토큰화

- 띄어쓰기 단위로 분리한다. 단, 동일한 의미라도 문장 부호 등에 따라 다른 토큰으로 처리될 수 있어 OOV 문제가 발생 가능하다.
- 직관적이지만 **cg**와 **cg.**를 서로 다른 토큰으로 인식하는 등의 문제가 발생한다.
- 같은 의미라도 다른 토큰으로 처리될 가능성이 높다.

2. 글자 토큰화

- 글자 단위로 분리한다.
- 어휘 사전이 작고 OOV 문제를 근본적으로 줄일 수 있다.
- 다만 토큰 자체가 가진 의미 정보가 부족하며, 한국어의 경우 자모 단위로 쪼개는 등의 추가적인 변형이 필요할 수 있다.

3. 형태소 토큰화

- 의미를 가진 최소 단위로 분리한다.

- 한국어와 같은 교착어 처리에 유리하며 문맥 이해가 가능하다.
- 주요 라이브러리는 **KoNLPy**, **NLTK**가 있다.

4. 하위 단어 토큰화

- 단어를 더 작은 하위 단어로 분리한다.
- **OOV** 문제와 신조어 문제를 효과적으로 완화한다.
- 알고리즘으로 **BPE**, **WordPiece**가 있다.

5. 임베딩

임베딩: 단어를 숫자 벡터로 표현하여 단어 간의 의미적 관계를 담아내는 기술

1. 전통적인 벡터화

- 원-핫 인코딩: 단어마다 고유 번호를 부여하고 해당 위치 인덱스만 1, 나머지는 0으로 채우는 방식이다.
- 빈도 벡터화: 해당 단어의 빈도를 벡터로 표시하는 방법
- **BoW**: 문장 내 단어의 빈도를 기록한다.
- 단어의 의미적 유사성을 담지 못하며, 벡터의 희소성이 커지는 차원의 저주 문제가 발생한다.

2. 워드 임베딩

- 단어를 밀집 벡터로 바꾸어 표현하며, 비슷한 의미를 가진 단어는 벡터 공간 상에서 비슷한 위치에 놓인다.
- **Word2Vec**, **fastText** 등이 있다.
- 단어를 고정된 값으로 표현하므로 문맥에 따라 의미가 달라지는 단어를 구분하기 어렵다는 단점이 있다.

3. 언어 모델

- 앞에 나온 단어들을 보고 다음에 어떤 단어가 올지 예측하는 모델
- 자기회귀 언어 모델: 앞에 나온 단어들을 바탕으로 다음 단어를 순서대로 예측하는 모델
- 통계적 언어 모델: 이전에 나온 단어들의 패턴과 확률을 이용해 다음 단어를 예측하는 모델
- 마르코프 체인: 현재 상태를 바탕으로 다음 상태를 확률적으로 예측하는 방법
- **N-gram**: 연속된 N개의 단어를 묶어서 다음 단어를 예측하는 가장 기본적인 언어 모델

4. TF-IDF

- 문서 빈도 $\times$ 역문서 빈도

- 단어 빈도만 보는 것이 아니라, 여러 문서에 공통적으로 등장하는 흔한 단어의 중요도는 낮추고, 특정 문서에만 등장하는 단어의 중요도는 높게 평가한다.
- 문장에서 어떤 단어가 핵심적인지 판별할 때 유용하다.
- 단어의 순서나 문맥적 의미를 직접 담고 있지는 않다.